

Contenido

Objetivo de la solución	2
Arquitectura general.....	2
Programación orientada a objetos.....	3
Jerarquía de productos	3
Interfaces	3
Persistencia con SQLite.....	4
Patrón DAO.....	4
Factory para DAOs	4
Strategy para pagos.....	5
Builder para órdenes	5
Facade para checkout	5
Servicios de negocio	6
Validaciones y excepciones	6
Menú por consola	6
Autenticación y permisos	7
Persistencia del carrito activo	7
Proyecto ejecutable	7
Conclusión.....	8

Objetivo de la solución

El sistema fue diseñado como una aplicación de consola en Java que modela el ciclo de vida completo de una compra en una plataforma de E-Commerce. La solución permite administrar usuarios, roles, productos, categorías, inventario, carrito, órdenes, pagos, envíos, seguimiento, reclamos, devoluciones, calificaciones y reportes. El diseño no se planteó como una suma de CRUDs independientes. La decisión central fue modelar el flujo real de negocio: el cliente consulta productos, administra un carrito, confirma una compra, procesa un pago, genera una orden, asocia un envío, descuenta stock y luego puede consultar seguimiento o iniciar procesos de post compra.

Arquitectura general

La arquitectura se organizó en capas para separar responsabilidades:

- **model:** entidades y reglas propias del dominio.
- **enums:** estados, roles y tipos cerrados del negocio.
- **interfaces:** contratos generales como Calculable, Mostrable, Descontable, Enviabile y ProcesadorPago.
- **dao:** contratos e implementaciones de persistencia.
- **database:** configuración e inicialización de SQLite.
- **service:** reglas de negocio y coordinación entre DAOs.
- **payment:** estrategias concretas de procesamiento de pago.
- **ui:** menús de consola e interacción con el usuario.
- **util:** validaciones reutilizables.
- **exception:** excepciones personalizadas del dominio.

Esta separación evita que el menú de consola contenga lógica de negocio o SQL. La UI solo solicita datos y delega en servicios. Los servicios validan reglas y coordinan operaciones. Los DAOs encapsulan el acceso a SQLite.

Programación orientada a objetos

La solución aplica **encapsulamiento** porque las entidades exponen métodos y validan sus cambios mediante setters o métodos específicos. Los atributos se mantienen privados o protegidos según corresponda. La abstracción se aplica principalmente en la clase **Producto**, definida como clase abstracta para representar los datos y comportamientos comunes de los productos. La herencia se aplica mediante **ProductoFísico**, **ProductoDigital** y **ProductoImportado**. Cada subtipo especializa el comportamiento de **Producto**. El polimorfismo se utiliza en dos puntos centrales: el cálculo de precio final de los productos y el procesamiento de pagos. En ambos casos, el comportamiento concreto depende del subtipo o de la estrategia seleccionada, sin concentrar toda la lógica en una única clase.

Jerarquía de productos

Producto fue definido como clase abstracta porque no representa un producto concreto instanciable, sino una base común. Las clases **ProductoFísico**, **ProductoDigital** y **ProductoImportado** permiten extender el sistema sin alterar la lógica general de productos. Esta decisión permite agregar nuevos tipos de producto en el futuro manteniendo el principio abierto/cerrado: se pueden crear nuevas subclases sin modificar el código cliente que trabaja contra **Producto**.

Interfaces

Se incorporaron interfaces para definir contratos de comportamiento:

- **Calculable**: objetos capaces de calcular totales.
- **Mostrable**: objetos capaces de mostrar información.
- **Descontable**: objetos capaces de aplicar descuentos.
- **Enviable**: objetos capaces de calcular costo de envío.
- **ProcesadorPago**: contrato para las estrategias de pago.

Estas interfaces desacoplan el comportamiento de las clases concretas y permiten aplicar polimorfismo.

Persistencia con SQLite

Se decidió utilizar **SQLite** porque la consigna lo contempla como implementación válida de persistencia. Además, **SQLite** es apropiado para una aplicación de consola porque no requiere un servidor externo, permite guardar datos entre ejecuciones y se integra con Java mediante **JDBC** (*Conectividad de Bases de Datos Java*). La base de datos se inicializa mediante un script **schema.sql** y se crea automáticamente al iniciar la aplicación si no existe. Se persisten usuarios, categorías, productos, inventario, carritos, órdenes, pagos, envíos, historial de envíos, reclamos, devoluciones y calificaciones.

Patrón DAO

Toda la persistencia se canaliza mediante interfaces **DAO** (Objeto de Acceso a Datos).

Las clases de servicio no dependen de **SQL** ni de detalles de **JDBC**, sino de contratos como *UsuarioDAO*, *ProductoDAO*, *CarritoDAO*, *OrdenDAO* o *ReclamoDAO*.

Las implementaciones SQLite se encargan de traducir objetos del dominio a registros de base de datos y viceversa. Esta decisión mejora la mantenibilidad porque permite cambiar la tecnología de persistencia sin modificar el dominio ni la UI.

Factory para DAOs

Se incorporó **DAOFactory** como fábrica abstracta y **SQLiteDAOFactory** como implementación concreta. La aplicación solicita **DAOs** a la fábrica sin instanciar directamente las clases **SQLite** desde cualquier parte del sistema.

Esta decisión centraliza la creación de dependencias de persistencia y cumple con el requisito de **Factory Method** o **Abstract Factory**.

Strategy para pagos

El procesamiento de pagos se resolvió mediante el patrón **Strategy**.

La interfaz **ProcesadorPago** define el contrato común y cada medio de pago posee su propia clase concreta:

- ProcesadorTarjetaCredito
- ProcesadorTarjetaDebito
- ProcesadorTransferenciaBancaria
- ProcesadorBilleteraVirtual
- ProcesadorPagoContraEntrega

La selección se realiza mediante **ProcesadorPagoFactory**. De esta manera se evita una solución basada en comparaciones de texto del tipo *if(tipoPago.equals("Tarjeta"))* y se mantiene una implementación polimórfica.

Builder para órdenes

OrdenCompraBuilder se utiliza para construir órdenes de compra de manera ordenada, evitando constructores extensos y facilitando la lectura del proceso de creación.

La orden requiere varios datos relacionados: cliente, fecha, ítems, total, estado, pago y envío. Builder permite configurar estos componentes de forma clara antes de generar el objeto final.

Facade para checkout

CheckoutFacade coordina el proceso completo de compra. Su responsabilidad es integrar carrito, validación de stock, procesamiento de pago, creación de envío, generación de orden, persistencia y descuento de inventario.

Esta decisión evita que el menú por consola o el servicio de órdenes deban conocer todos los pasos internos del checkout. El proceso queda encapsulado detrás de un método de alto nivel: *confirmarCompra*.

Servicios de negocio

- validación de permisos por rol;
- validación de usuarios activos;
- control de emails duplicados;
- control de productos y categorías duplicadas;
- validación de stock;
- operaciones de inventario;
- confirmación de compras;
- actualización de estados;
- generación de reclamos, devoluciones y calificaciones;
- generación de reportes.

Esto permite mantener el menú de consola simple y delegar la lógica de negocio en clases especializadas.

Validaciones y excepciones

Las reglas generales de validación se centralizaron en **ValidadorDominio**.

Se implementaron excepciones personalizadas para representar errores propios del sistema, como **ProductoNoEncontradoException**, **StockInsuficienteException**, **CarritoVacioException**, **PagoRechazadoException** y **PermisoDenegadoException**.

Las excepciones extienden de **EcommerceException**, que a su vez extiende **RuntimeException**. Esto evita llenar el dominio con firmas throws innecesarias y mantiene un manejo uniforme de errores de negocio.

Menú por consola

La interfaz de usuario se implementó mediante menús separados por módulo. **MenuPrincipal** funciona como punto de acceso a cada sección y aplica control de permisos según el rol del usuario autenticado.

Cada submenú se limita a leer datos, mostrar información y llamar servicios. Esta decisión evita mezclar presentación, reglas de negocio y persistencia.

Autenticación y permisos

Se agregó inicio de sesión por email y contraseña, creación del primer administrador y sesión de usuario activa.

El acceso a módulos se controla según rol:

- Cliente: carrito, órdenes, seguimiento y post compra.
- Administrador: gestión completa y reportes.
- Operador de Ventas: órdenes, pagos, seguimiento y post compra.
- Responsable de Logística: envíos y seguimiento.

Esto permite cumplir la consigna de acciones restringidas por permisos.

Persistencia del carrito activo

Durante la revisión final se detectó que el carrito estaba gestionado principalmente en memoria. Para mejorar el cumplimiento del requisito de persistencia entre ejecuciones, se agregó **CarritoDAO** y **SQLiteCarritoDAO**.

Ahora el carrito activo y sus ítems se persisten en las tablas carritos y carrito_items. Esta mejora hace que el módulo de carrito sea más consistente con el resto del sistema y con la exigencia de no depender exclusivamente de memoria.

Proyecto ejecutable

Se agregó configuración de maven-shade-plugin para generar un JAR ejecutable con clase principal ecommerce.Main. Esto facilita la entrega como proyecto ejecutable, además del código fuente.

Conclusión

La solución cumple con los conceptos principales de Programación Avanzada: orientación a objetos, herencia, polimorfismo, interfaces, colecciones, excepciones, persistencia, DAO, Factory, Strategy, Builder, Facade, validaciones, UML y menú por consola.

El diseño prioriza separación de responsabilidades, mantenibilidad y modelado de un proceso real de negocio.

La aplicación no se limita a CRUDs aislados, sino que integra los módulos mediante el flujo Cliente → Carrito → Checkout → Pago → Orden → Envío → Seguimiento → Post compra → Reportes.